```
/* ============================================================
 EXCEPTION HANDLING IN ORACLE PL/SQL

   Topics:
   1. Oracle Predefined Exceptions
      - NO_DATA_FOUND
      - TOO_MANY_ROWS
      - ZERO_DIVIDE
      - INVALID_NUMBER
      - VALUE_ERROR

   2. User Defined Exceptions
      - Creating Custom Exceptions
      - Raising Exceptions

   3. Exception Functions
      - SQLCODE
      - SQLERRM

   Labs:
   - ATM Withdrawal Validation
   - Product Stock Validation
   - Error Logging Example
   ============================================================ */

SET SERVEROUTPUT ON SIZE UNLIMITED;

/* ============================================================
   1. DROP OLD TABLES
   ============================================================ */

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE bank_accounts';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE products';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE error_logs';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/
```

```
/* ============================================================
   2. CREATE TABLES
   ============================================================ */

CREATE TABLE bank_accounts
(
    account_id      NUMBER PRIMARY KEY,
    account_name    VARCHAR2(100),
    balance         NUMBER,
    account_status  VARCHAR2(20)
);

CREATE TABLE products
(
    product_id      NUMBER PRIMARY KEY,
    product_name    VARCHAR2(100),
    stock_quantity  NUMBER,
    unit_price      NUMBER
);

CREATE TABLE error_logs
(
    log_id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    error_date      DATE DEFAULT SYSDATE,
    program_name    VARCHAR2(100),
    error_code      NUMBER,
    error_message   VARCHAR2(4000)
);

/* ============================================================
   3. INSERT SAMPLE DATA
   ============================================================ */

INSERT INTO bank_accounts
(account_id, account_name, balance, account_status)
VALUES
(1, 'Aung Aung', 1000000, 'ACTIVE');

INSERT INTO bank_accounts
(account_id, account_name, balance, account_status)
VALUES
(2, 'Mg Mg', 500000, 'ACTIVE');

INSERT INTO bank_accounts
(account_id, account_name, balance, account_status)
VALUES
(3, 'Su Su', 200000, 'BLOCKED');

INSERT INTO products
(product_id, product_name, stock_quantity, unit_price)
VALUES
(1, 'Laptop', 10, 1500000);

INSERT INTO products
```

```
(product_id, product_name, stock_quantity, unit_price)
VALUES
(2, 'Mouse', 50, 15000);

INSERT INTO products
(product_id, product_name, stock_quantity, unit_price)
VALUES
(3, 'Keyboard', 20, 30000);

COMMIT;

/* ============================================================
   PART A: ORACLE PREDEFINED EXCEPTIONS
   ============================================================ */

/* ============================================================
   4. NO_DATA_FOUND Example

   This exception occurs when SELECT INTO returns no row.
   ============================================================ */
```

```
SQL> DECLARE
    v_account_name bank_accounts.account_name%TYPE;
BEGIN
    SELECT account_name
    INTO v_account_name
    FROM bank_accounts
    WHERE account_id = 999;

    DBMS_OUTPUT.PUT_LINE('Account Name: ' || v_account_name);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('NO_DATA_FOUND: No account found with this ID.');
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14   15  NO_DATA_FOUND:
No account found with this ID.

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   5. TOO_MANY_ROWS Example

   This exception occurs when SELECT INTO returns more than one row.
   ============================================================ */
```

```
SQL> DECLARE
    v_account_name bank_accounts.account_name%TYPE;
BEGIN
    SELECT account_name
    INTO v_account_name
    FROM bank_accounts
    WHERE account_status = 'ACTIVE';

    DBMS_OUTPUT.PUT_LINE('Account Name: ' || v_account_name);

EXCEPTION  2
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('TOO_MANY_ROWS: Query returned more than one row.');
END;
/
  3    4    5    6    7    8    9   10   11   12   13   14   15  TOO_MANY_ROWS: Query
 returned more than one row.

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   6. ZERO_DIVIDE Example

   This exception occurs when a number is divided by zero.
   ============================================================ */
```

```
SQL> DECLARE
    v_total_amount NUMBER := 100000;
    v_count        NUMBER := 0;
    v_average      NUMBER;
BEGIN
    v_average := v_total_amount / v_count;

    DBMS_OUTPUT.PUT_LINE('Average Amount: ' || v_average);

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('ZERO_DIVIDE: Cannot divide by zero.');
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14  ZERO_DIVIDE: Cannot
divide by zero.

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   7. INVALID_NUMBER Example

   This exception occurs when character data cannot be converted to
number.
   ============================================================ */
```

```
SQL> DECLARE
    v_text_number VARCHAR2(20) := 'ABC123';
    v_number      NUMBER;
BEGIN
    v_number := TO_NUMBER(v_text_number);

    DBMS_OUTPUT.PUT_LINE('Converted Number: ' || v_number);

EXCEPTION
    WHEN INVALID_NUMBER THEN
        DBMS_OUTPUT.PUT_LINE('INVALID_NUMBER: Text cannot be converted to number.');
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13  DECLARE
*
ERROR at line 1:
ORA-06502: PL/SQL: numeric or value error: character to number conversion error
ORA-06512: at line 5
```

```
/* ============================================================
   8. VALUE_ERROR Example

   This exception occurs when value size or type is invalid.
   Example: assigning long text into short variable.
   ============================================================ */
```

```
SQL> DECLARE
    v_short_text VARCHAR2(5);
BEGIN
    v_short_text := 'ORACLE DATABASE';

    DBMS_OUTPUT.PUT_LINE('Text: ' || v_short_text);

EXCEPTION
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('VALUE_ERROR: Value is too large for the variable.');
END;
/
  2    3    4    5    6    7    8    9   10   11   12  VALUE_ERROR: Value is too large
for the variable.

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   PART B: USER DEFINED EXCEPTIONS
   ============================================================ */

/* ============================================================
   9. Creating and Raising Custom Exception

   Requirement:
   - If withdrawal amount is greater than balance,
     raise custom exception.
   ============================================================ */
```

```
SQL> DECLARE
    v_balance           NUMBER := 300000;
    v_withdraw_amount   NUMBER := 500000;

    e_insufficient_balance EXCEPTION;
BEGIN
    IF v_withdraw_amount > v_balance THEN
        RAISE e_insufficient_balance;
    END IF;

    v_balance := v_balance - v_withdraw_amount;

    DBMS_OUTPUT.PUT_LINE('Withdrawal successful.');
    DBMS_OUTPUT.PUT_LINE('Remaining Balance: ' || v_balance);

EXCEPTION
    WHEN e_insufficient_balance THEN
        DBMS_OUTPUT.PUT_LINE('Custom Exception: Insuffici  2  ent balance.');
END;
/
  3    4    5    6    7    8    9   10   11   12   13   14   15   16   17   18   19   20
Custom Exception: Insufficient balance.

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   10. Custom Exception with Business Rule

   Requirement:
   - Account must be ACTIVE.
   - If account is BLOCKED, raise exception.
   ============================================================ */
```

```
SQL> DECLARE
    v_account_status bank_accounts.account_status%TYPE;

    e_account_blocked EXCEPTION;
BEGIN
    SELECT account_status
    INTO v_account_status
    FROM bank_accounts
    WHERE account_id = 3;

    IF v_account_status = 'BLOCKED' THEN
        RAISE e_account_blocked;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Account is active. Transaction allowed.');

EXCEPTION
    WHEN e_account_blocked THEN
        DBMS_OUTPUT.PUT_LINE('Custom Exception: Account is blocked.');
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Account not found.');
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14   15   16   17   18   19
 20   21   22   23  Custom Exception: Account is blocked.

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   PART C: SQLCODE AND SQLERRM
   ============================================================ */

/* ============================================================
   11. SQLCODE and SQLERRM Example

   SQLCODE returns Oracle error number.
   SQLERRM returns Oracle error message.
   ============================================================ */
```

```
SQL> DECLARE
    v_number NUMBER;
BEGIN
    v_number := TO_NUMBER('ABC');

    DBMS_OUTPUT.PUT_LINE('Number: ' || v_number);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error Code: ' || SQLCODE);
        DBMS_OUTPUT.PUT_LINE('Error Message: ' || SQLERRM);
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13  Error Code: -6502
Error Message: ORA-06502: PL/SQL: numeric or value error: character to number
conversion error

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   12. Error Logging Example

   Store error details into error_logs table.
   ============================================================ */
```

```
SQL> DECLARE
    v_result NUMBER;
    v_code   NUMBER;
    v_msg    VARCHAR2(4000);
BEGIN
    v_result := 10 / 0;

    DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);

EXCEPTION
    WHEN OTHERS THEN
        v_code := SQLCODE;
        v_msg  := SQLERRM;

        INSERT INTO error_logs
        (
            program_name,
            error_code,
            error_message
        )
        VALUES
        (
            'Error Logging Example',
            v_code,
            v_msg
        );

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Error saved into error_logs table.');
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14   15   16   1
7   18   19   20   21   22   23   24   25   26   27   28   29   30   31   32
 Error saved into error_logs table.

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   LAB 1: ATM Withdrawal Validation
   ============================================================ */

DECLARE
    v_account_id        bank_accounts.account_id%TYPE := 1;
    v_withdraw_amount   NUMBER := 300000;
    v_balance           bank_accounts.balance%TYPE;
    v_status            bank_accounts.account_status%TYPE;

    e_insufficient_balance EXCEPTION;
```

```
    e_account_blocked      EXCEPTION;
    e_invalid_amount       EXCEPTION;
BEGIN
    SELECT balance, account_status
    INTO v_balance, v_status
    FROM bank_accounts
    WHERE account_id = v_account_id;

    IF v_withdraw_amount <= 0 THEN
        RAISE e_invalid_amount;
    END IF;

    IF v_status <> 'ACTIVE' THEN
        RAISE e_account_blocked;
    END IF;

    IF v_withdraw_amount > v_balance THEN
        RAISE e_insufficient_balance;
    END IF;

    UPDATE bank_accounts
    SET balance = balance - v_withdraw_amount
    WHERE account_id = v_account_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('ATM withdrawal successful.');
    DBMS_OUTPUT.PUT_LINE('Withdraw Amount: ' || v_withdraw_amount);
    DBMS_OUTPUT.PUT_LINE('Remaining Balance: ' || (v_balance -
v_withdraw_amount));

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ATM Error: Account not found.');

    WHEN e_invalid_amount THEN
        DBMS_OUTPUT.PUT_LINE('ATM Error: Withdraw amount must be greater
than zero.');

    WHEN e_account_blocked THEN
        DBMS_OUTPUT.PUT_LINE('ATM Error: Account is blocked.');

    WHEN e_insufficient_balance THEN
        DBMS_OUTPUT.PUT_LINE('ATM Error: Insufficient balance.');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected ATM Error: ' || SQLERRM);
END;
/
```

```
ATM withdrawal successful.
Withdraw Amount: 300000
Remaining Balance: 700000

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   LAB 2: Product Stock Validation
   ============================================================ */

DECLARE
    v_product_id       products.product_id%TYPE := 1;
    v_order_quantity   NUMBER := 3;
    v_stock_quantity   products.stock_quantity%TYPE;

    e_out_of_stock     EXCEPTION;
    e_invalid_quantity EXCEPTION;
BEGIN
    SELECT stock_quantity
    INTO v_stock_quantity
    FROM products
    WHERE product_id = v_product_id;

    IF v_order_quantity <= 0 THEN
        RAISE e_invalid_quantity;
    END IF;

    IF v_order_quantity > v_stock_quantity THEN
        RAISE e_out_of_stock;
    END IF;

    UPDATE products
    SET stock_quantity = stock_quantity - v_order_quantity
    WHERE product_id = v_product_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Product stock updated successfully.');
    DBMS_OUTPUT.PUT_LINE('Order Quantity: ' || v_order_quantity);
    DBMS_OUTPUT.PUT_LINE('Remaining Stock: ' || (v_stock_quantity -
v_order_quantity));

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Stock Error: Product not found.');

    WHEN e_invalid_quantity THEN
        DBMS_OUTPUT.PUT_LINE('Stock Error: Order quantity must be greater
than zero.');

    WHEN e_out_of_stock THEN
        DBMS_OUTPUT.PUT_LINE('Stock Error: Not enough stock.');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected Stock Error: ' || SQLERRM);
END;
/
```

```
Product stock updated successfully.
Order Quantity: 3
Remaining Stock: 7

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   LAB 3: Error Logging Example
   ============================================================ */

DECLARE
    v_account_name bank_accounts.account_name%TYPE;
    v_code   NUMBER;
    v_msg    VARCHAR2(4000);
BEGIN
    SELECT account_name
    INTO v_account_name
    FROM bank_accounts
    WHERE account_id = 9999;

    DBMS_OUTPUT.PUT_LINE('Account Name: ' || v_account_name);

EXCEPTION
    WHEN OTHERS THEN
        v_code := SQLCODE;
        v_msg  := SQLERRM;

        INSERT INTO error_logs
        (
            program_name,
            error_code,
            error_message
        )
        VALUES
        (
            'LAB 3: Error Logging Example',
            v_code,
            v_msg
        );

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Error occurred and saved into error_logs
table.');
        DBMS_OUTPUT.PUT_LINE('Error Code: ' || SQLCODE);
        DBMS_OUTPUT.PUT_LINE('Error Message: ' || SQLERRM);
END;
/
```

```
 Error occurred and saved into error_logs table.
Error Code: 100
Error Message: ORA-01403: no data found

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   FINAL CHECKING
   ============================================================ */
```

```
SQL> SELECT * FROM bank_accounts ORDER BY account_id;

ACCOUNT_ID
----------
ACCOUNT_NAME
--------------------------------------------------
   BALANCE ACCOUNT_STATUS
---------- --------------------
         1
Aung Aung
    700000 ACTIVE

         2
Mg Mg
    500000 ACTIVE

ACCOUNT_ID
----------
ACCOUNT_NAME
--------------------------------------------------
   BALANCE ACCOUNT_STATUS
---------- --------------------

         3
Su Su
    200000 BLOCKED
```

```
SQL> SELECT * FROM products ORDER BY product_id;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
STOCK_QUANTITY UNIT_PRICE
-------------- ----------
         1
Laptop
             7    1500000

         2
Mouse
            50      15000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
STOCK_QUANTITY UNIT_PRICE
-------------- ----------

         3
Keyboard
            20      30000
```

```
SQL> SELECT * FROM error_logs ORDER BY log_id;

    LOG_ID ERROR_DAT
---------- ---------
PROGRAM_NAME
--------------------------------------------------
ERROR_CODE
----------
ERROR_MESSAGE
--------------------------------------------------
         1 21-JUN-26
Error Logging Example
     -1476
ORA-01476: divisor is equal to zero


    LOG_ID ERROR_DAT
---------- ---------
PROGRAM_NAME
--------------------------------------------------
ERROR_CODE
----------
ERROR_MESSAGE
--------------------------------------------------
         2 21-JUN-26
LAB 3: Error Logging Example
       100
ORA-01403: no data found
```